R - EXERCISE OVERVIEW

Week 4 & 5

- ***16 – Data wrangling techniques***

  No exercises.

- ***17 – Selecting rows***

  Use again the dataset `transactions.csv` (stored as `myData`).

  1. Select rows 10 to 20.
  2. Select all purchases from 2010.
  3. Select all purchases with purchase amount greater than 100 which were made from 01.01.2009 onwards.

- ***18 – Selecting columns***

  Use again the dataset `transactions.csv` (stored as `myData`).

  1. From `myData`, select the following columns: `Customer`, `TransDate`, and `Quantity` and store them in a new object called `myData2`.
  2. From `myData`, select
     a. the following columns: `Customer`, `TransDate`, and `Quantity` and,
     b. at the same time, select all rows with a purchase amount between 100 and 200.

     Store this subset in a new object called `myData3`.

- ***19 – Updating rows and columns***

  Use again the dataset `transactions.csv` (stored as `myData`).

  1. From `myData`, create a new column calculating the difference between `PurchAmount` and `Cost`. Call it `Profit`.
  2. Rename `Profit` to `ProfitChange`.
  3. Delete `ProfitChange` again.

- ***20 – Basic aggregating techniques***

  Use again the dataset `transactions.csv` (stored as `myData`).

1. Sum `PurchAmount` by `Customer` and `TransDate` and save it as a column named `AggDayPurch`. Use the := operator and not the = operator.
2. Create a new data table that includes the Customer ID (`Customer`) as well as the new variable `PurchFreq`, i.e., the number of transactions by customer.
   Hint: Use `.N`.
3. Count the overall number of transactions.

- ***21 – Advanced aggregating techniques***

  Use again the dataset `transactions.csv` (stored as `myData`).

  1. Aggregate the purchase amount of all transactions per customer on a yearly basis for year 2007 and 2008. Use the aggregating function `sum()`.
  2. How many customers purchased for more than $50 in total between 2008 and 2009?

- ***22 – Combined select-aggregate operations***

  Use again the dataset `transactions.csv` (stored as `myData`).

  1. Create a new column in your data table and store, for each customer and transaction, the cost of the previous purchase as an additional variable*: You can do this by creating an aggregated lag-shifted variable based on the `Cost` column. Use an offset of 1 and aggregate the data by customer. You can name the resulting column `CostLag`.*

- ***23 – Basic merging techniques***

  No exercises.

- ***24 – Inner joins and full outer joins***

  Use again the dataset `transactions.csv` (stored as `myData`).
  In addition, use the dataset `demographics.csv`. This dataset contains information on customers' gender, birthdate, zip code and join date.

  In addition to `transaction.csv` (stored as `myData`), read in demographics.csv and call it `demographics`. Make sure `JoinDate` is in the right format.

  1. Merge the tables `transactions` and `demographics` by the column `Customer` using an outer left join.
  2. Merge the tables `transactions` and `demographics` by the column `Customer` using an inner join. Do this only for the customers born after 1980.

- ***25 – Left and right outer joins***

Continue using the dataset `transactions.csv` (stored as `myData`) and the dataset `demographics.csv` (stored as `demographics`).

1. Merge transaction and demographics by `Customer` using an outer left join.
2. Merge transaction and demographics by `Customer` using an outer right join.

- ***26 – Advanced merging techniques***

Continue using the dataset `transactions.csv` (stored as `myData`) and the dataset `demographics.csv` (stored as `demographics`).

1. Extract the "`year`" information from the columns from `TransDate` and `JoinDate` and save them as `TransYear` and `JoinYear`. Then, merge the datasets `transaction` and `demographics` by (`Customer` and `TransYear`) and (`Customer` and `JoinYear`) using an inner join.